

```

unsigned long dPartitionInit(unsigned long start, unsigned long totalSize)
{
    /*功能：初始化内存。
    1. 在地址start处，首先是要有dPartition结构体表示整个数据结构(也即句柄)。
    2. 然后，一整块的EMB被分配（以后使用内存会逐渐拆分），在内存中紧紧跟在dP后面，然后dP的
    firstFreeStart指向EMB。
    3. 返回start首地址(也即句柄)。
    注意有两个地方的大小问题：
    第一个是由于内存肯定要有有一个EMB和一个dPartition，totalSize肯定要比这两个加起来大。
    第二个注意EMB的size属性不是totalSize，因为dPartition和EMB自身都需要要占空间。
    */
    totalSize = ALIGN8(totalSize);

    // 检查总大小是否符合要求
    if (totalSize < dPartition_size + EMB_size)
    {
        return 0;
    }

    struct dPartition *dp = (struct dPartition *)start;
    dp->size = totalSize;

    unsigned long embAddr = start + dPartition_size;
    struct EMB *emb = (struct EMB *)embAddr;
}

```

```

emb->size = ALIGN8(totalSize - dPartition_size - EMB_size);
emb->nextStart = 0;

dp->firstFreeStart = embAddr;

return start;
}

void dPartitionwalkByAddr(unsigned long dp){
    /*功能：本函数遍历输出EMB 方便调试
    1. 先打印dp的信息，可调用上面的showdPartition。
    2. 然后按地址的大小遍历EMB，对于每一个EMB，可以调用上面的showEMB输出其信息

    */
    struct dPartition *pdp = (struct dPartition *)dp;
    unsigned long curr;

    showdPartition(pdp);

    curr = pdp->firstFreeStart;
    while (curr)
    {
        showEMB((struct EMB *)curr);
        curr = ((struct EMB *)curr)->nextStart;
    }
}

unsigned long dPartitionAllocFirstFit(unsigned long dp, unsigned long size)
{
    /*功能：分配一个空间
    1. 使用firstfit的算法分配空间，
    2. 成功分配返回首地址，不成功返回0
    3. 从空闲内存块组成的链表中拿出一块供我们来分配空间(如果提供给分配空间的内存块空间大于
    size，我们还将把剩余部分放回链表中)，并维护相应的空闲链表以及句柄
    注意的地方：
        1.EMB类型的数据的存在本身就占用了一定的空间。

    */
    // 对齐
    size = ALIGN8(size);
    struct dPartition *pdp = (struct dPartition *)dp;
    unsigned long prev = 0;
    unsigned long curr = pdp->firstFreeStart;

    while (curr)
    {
        struct EMB *pemb = (struct EMB *)curr;
        if (pemb->size >= size)
        {
            // 实际可用内存的起始地址
            unsigned long userAddr = curr + EMB_size;
            // 空闲块剩余的大小
            unsigned long remain = pemb->size - size;

            // 如果剩余空间足够放下一个新的EMB块头，则进行分割

```

```

    if (remain > EMB_size)
    {
        // 新建一个EMB
        unsigned long next = pemb->nextStart;
        unsigned long splitAddr = curr + EMB_size + size;
        struct EMB *newEmb = (struct EMB *)splitAddr;
        newEmb->size = ALIGN8(remain - EMB_size);
        newEmb->nextStart = next;

        // 将前一个空闲块的next设置为新的空闲块地址
        if (prev)
            ((struct EMB *)prev)->nextStart = splitAddr;
        else
            pdp->firstFreeStart = splitAddr;

        // 更新当前块大小为请求大小
        pemb->size = size;
    }
    else
    {
        // 剩余空间不足以存放一个新的EMB，整体分配此块
        if (prev)
            ((struct EMB *)prev)->nextStart = pemb->nextStart;
        else
            pdp->firstFreeStart = pemb->nextStart;
    }

    return userAddr;
}
prev = curr;
curr = pemb->nextStart;
}
return 0;
}

unsigned long dPartitionFreeFirstFit(unsigned long dp, unsigned long start){
    /*功能：释放一个空间
    1. 按照对应的fit的算法释放空间
    2. 注意检查要释放的start~end这个范围是否在dp有效分配范围内
        返回1 没问题
        返回0 error
    3. 需要考虑两个空闲且相邻的内存块的合并

    */
    struct dPartition *pdp = (struct dPartition *)dp;
    if (!start) return 0;
    // EMB 头部地址
    unsigned long embAddr = start - EMB_size;
    // 分区范围
    unsigned long partStart = dp;
    unsigned long partEnd = dp + pdp->size;
    // 检查dp是否在有效分配范围内
    if (embAddr < partStart + dPartition_size || embAddr + EMB_size + ((struct
    EMB *)embAddr)->size > partEnd)
        return 0;

```

```

struct EMB *freed = (struct EMB *)embAddr;
unsigned long prev = 0;
unsigned long curr = pdp->firstFreeStart;

// 找到插入位置
while (curr && curr < embAddr)
{
    prev = curr;
    curr = ((struct EMB *)curr)->nextStart;
}
// 将 freed 插入
freed->nextStart = curr;
if (prev)
    ((struct EMB *)prev)->nextStart = embAddr;
else
    pdp->firstFreeStart = embAddr;

// 与后续块合并
if (curr)
{
    unsigned long freedEnd = embAddr + EMB_size + freed->size;
    if (freedEnd == curr)
    {
        freed->size += EMB_size + ((struct EMB *)curr)->size;
        freed->nextStart = ((struct EMB *)curr)->nextStart;
        curr = freed->nextStart;
    }
}

// 与前驱块合并
if (prev)
{
    unsigned long prevEnd = prev + EMB_size + ((struct EMB *)prev)->size;
    if (prevEnd == embAddr)
    {
        ((struct EMB *)prev)->size += EMB_size + freed->size;
        ((struct EMB *)prev)->nextStart = freed->nextStart;
    }
}

return 1;
}

```

2.eFPartition.c

```

void eFPartitionWalkByAddr(unsigned long efpHandler){
    // TODO
    /*功能：本函数是为了方便查看和调试的。
    1. 打印eFPartiiton结构体的信息，可以调用上面的showeFPartition函数。
    2. 遍历每一个EEB，打印出他们的地址以及下一个EEB的地址（可以调用上面的函数showEEB）

    */
    struct eFPartition *efp = (struct eFPartition *)efpHandler;
    unsigned long curr;

```

```

showeFPartition(efp);

curr = efp->firstFree;
while (curr) {
    showEEB((struct EEB *)curr);
    curr = ((struct EEB *)curr)->next_start;
}
}

unsigned long eFPartitionTotalSize(unsigned long perSize, unsigned long n){
    // TODO
    /*功能：计算占用空间的实际大小，并将这个结果返回
    1. 根据参数perSize（每个大小）和n个数计算总大小，注意perSize的对齐。
        例如perSize是31字节，你想8字节对齐，那么计算大小实际代入的一个块的大小就是32字节。
    2. 同时还需要注意“隔离带”EEB的存在也会占用4字节的空间。
        typedef struct EEB {
            unsigned long next_start;
        }EEB;
    3. 最后别忘记加上eFPartition这个数据结构的大小，因为它也占一定的空间。

    */
    unsigned long perSize_aligned = ALIGN8(perSize);
    return ALIGN8(eFPartition_size) + n * (EEB_size + perSize_aligned);
}

unsigned long eFPartitionInit(unsigned long start, unsigned long perSize,
unsigned long n){
    // TODO
    /*功能：初始化内存
    1. 需要创建一个eFPartition结构体，需要注意的是结构体的perSize不是直接传入的参数
    perSize，需要对齐。结构体的next_start也需要考虑一下其本身的大小。
    2. 就是先把首地址start开始的一部分空间作为存储eFPartition类型的空间
    3. 然后再对除去eFPartition存储空间后的剩余空间开辟若干连续的空闲内存块，将他们连起来构成一个链。注意最后一块的EEB的nextstart应该是0
    4. 需要返回一个句柄，也即返回eFPartition *类型的数据
    注意的地方：
        1.EEB类型的数据的存在本身就占用了一定的空间。
    */

    unsigned long perSize_aligned = ALIGN8(perSize);
    struct eFPartition *efp = (struct eFPartition *)start;
    unsigned long base = start + ALIGN8(eFPartition_size);
    unsigned long i, addr;

    efp->totalN    = n;
    efp->perSize    = perSize_aligned;
    efp->firstFree = base;

    // 建立链表
    for (i = 0; i < n; i++)
    {
        addr = base + i * (EEB_size + perSize_aligned);
        if (i + 1 < n)
            ((struct EEB *)addr)->next_start = addr + EEB_size + perSize_aligned;
    }
}

```

```

        else
            ((struct EEB *)addr)->next_start = 0;
    }
    return start;
}

unsigned long eFPartitionAlloc(unsigned long EFPHandler){
    // TODO
    /*功能：分配一个空间
    1. 本函数分配一个空闲块的内存并返回相应的地址，EFPHandler表示整个内存的首地址
    2. 事实上EFPHandler就是我们的句柄，EFPHandler作为eFPartition *类型的数据，其存放了我们需要的firstFree数据信息
    3. 从空闲内存块组成的链表中拿出一块供我们来分配空间，并维护相应的空闲链表以及句柄
    注意的地方：
        1.EEB类型的数据的存在本身就占用了一定的空间。

    */
    struct eFPartition *efp = (struct eFPartition *)EFPHandler;
    unsigned long block = efp->firstFree;
    if (!block) return 0;
    // 更新firstFree
    efp->firstFree = ((struct EEB *)block)->next_start;
    return block + EEB_size;
}

unsigned long eFPartitionFree(unsigned long EFPHandler,unsigned long mbStart){
    // TODO
    /*功能：释放一个空间
    1. mbstart将成为第一个空闲块，EFPHandler的firstFree属性也需要相应大的更新。
    2. 同时我们也需要更新维护空闲内存块组成的链表。

    */
    struct eFPartition *efp = (struct eFPartition *)EFPHandler;
    unsigned long block = mbStart - EEB_size;
    if (block < EFPHandler + ALIGN8(eFPartition_size)) return 0;
    ((struct EEB *)block)->next_start = efp->firstFree;
    // mbstart将成为第一个空闲块
    efp->firstFree = block;
    return 1;
}

```

3. pMemInit.c

```

void memTest(unsigned long start, unsigned long grainSize){
    // TODO
    /*功能：检测算法
    这一个函数对应实验讲解ppt中的第一大功能-内存检测。
    本函数的功能是检测从start开始有多大的内存可用，具体算法参照ppt检测算法的文字描述
    注意点三个：
    1、覆盖写入和读出就是往指针指向的位置写和读，不要想复杂。
        （至于为什么这种检测内存的方法可行大家要自己想一下）
    2、开始的地址要大于1M，需要做一个if判断。
    3、grainSize不能太小，也要做一个if判断
    */

```

```

/*
从start开始，以grainSize为步长，进行内存检测
检测方法：
1) 读出grain的头2个字节
2) 覆盖写入0xAA55，再读出并检查是否是0xAA55，若不是则检测结束；
3) 覆盖写入0x55AA，再读出并检查是否是0x55AA，若不是则检测结束；
4) 写回原来的值
5) 对grain的尾2个字节，重复2-4
6) 步进到下一个grain，重复1-5，直到检测结束
*/
if (start < 0x100000 || grainSize < sizeof(unsigned short))
{
    pMemStart = pMemSize = 0;
    myPrintk(0x7, "MemStart: %x \n", pMemStart);
    myPrintk(0x7, "MemSize: %x \n", pMemSize);
    return;
}

unsigned long addr = start;
unsigned long size = 0;

while (1)
{
    // 检测 head 两字节
    unsigned short *head = (unsigned short *)addr;
    unsigned short oldHead = *head;
    *head = 0xAA55;
    if (*head != 0xAA55) break;
    *head = 0x55AA;
    if (*head != 0x55AA) break;
    *head = oldHead;

    // 检测 tail 两字节
    unsigned short *tail = (unsigned short *)(addr + grainSize -
sizeof(unsigned short));
    unsigned short oldTail = *tail;
    *tail = 0xAA55;
    if (*tail != 0xAA55) break;
    *tail = 0x55AA;
    if (*tail != 0x55AA) break;
    *tail = oldTail;

    // 下一个 grain
    size += grainSize;
    addr += grainSize;
}

pMemStart = start;
pMemSize = size;
myPrintk(0x7, "MemStart: %x \n", pMemStart);
myPrintk(0x7, "MemSize: %x \n", pMemSize);
}

extern unsigned long _end;
void pMemInit(void){
    unsigned long _end_addr = (unsigned long) &_end;

```

```

memTest(0x100000,0x1000);
myPrintk(0x7,"_end: %x \n", _end_addr);
if (pMemStart <= _end_addr) {
    pMemSize -= _end_addr - pMemStart;
    pMemStart = _end_addr;
}

// TODO:此处选择不同的内存管理算法
pMemHandler = dPartitionInit(pMemStart,pMemSize);
}

```

4.addNewCmd

```

void addNewCmd( unsigned char *cmd,
               int (*func)(int argc, unsigned char **argv),
               void (*help_func)(void),
               unsigned char* description){
    // TODO
    /*功能：增加命令
    1. 使用malloc创建一个cmd的结构体，新增命令。
    2. 同时还需要维护一个表头为ourCmds的链表。
    */
    struct cmd *newCmd = malloc(cmd_size);
    if (newCmd == NULL) return;

    unsigned char *p = (unsigned char *)newCmd;
    for (int i = 0; i < cmd_size; i++)
    {
        p[i] = 0;
    }

    //逐字拷贝
    int i = 0;
    while (cmd[i] && i < sizeof(newCmd->cmd) - 1)
    {
        newCmd->cmd[i] = cmd[i];
        i++;
    }
    newCmd->cmd[i] = '\0';

    i = 0;
    while (description[i] && i < sizeof(newCmd->description) - 1)
    {
        newCmd->description[i] = description[i];
        i++;
    }
    newCmd->description[i] = '\0';

    newCmd->func      = func;
    newCmd->help_func = help_func;
    newCmd->nextCmd   = NULL;

    if (ourCmds)
    {
        struct cmd *tmp = ourCmds;

```



```

        while (tmp->nextCmd)
        {
            tmp = tmp->nextCmd;
        }
        tmp->nextCmd = newCmd;
    }
    else
    {
        ourCmds = newCmd;
    }
}

```

第三部分 编译过程进行说明

src文件夹结构如下：

```

liyi@MSI:~/OSLab/Lab4$ tree
.
├─ Makefile
├─ Makefile:Zone.Identifier
├─ multibootheader
│   ├── multibootHeader.S
│   └─ multibootHeader.S:Zone.Identifier
├─ myOS
│   ├── Makefile
│   ├── Makefile:Zone.Identifier
│   ├── dev
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── i8253.c
│   │   ├── i8253.c:Zone.Identifier
│   │   ├── i8259A.c
│   │   ├── i8259A.c:Zone.Identifier
│   │   ├── uart.c
│   │   ├── uart.c:Zone.Identifier
│   │   ├── vga.c
│   │   └─ vga.c:Zone.Identifier
│   ├── i386
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── io.c
│   │   ├── io.c:Zone.Identifier
│   │   ├── irq.S
│   │   ├── irq.S:Zone.Identifier
│   │   ├── irq.c
│   │   └─ irq.c:Zone.Identifier
│   └─ include
│       ├── i8253.h
│       ├── i8253.h:Zone.Identifier
│       ├── i8259.h
│       ├── i8259.h:Zone.Identifier
│       ├── io.h
│       ├── io.h:Zone.Identifier
│       └─ irq.h

```

```

| | └─ irq.h:Zone.Identifier
| | └─ kmalloc.h
| | └─ kmalloc.h:Zone.Identifier
| | └─ malloc.h
| | └─ malloc.h:Zone.Identifier
| | └─ mem.h
| | └─ mem.h:Zone.Identifier
| | └─ myPrintk.h
| | └─ myPrintk.h:Zone.Identifier
| | └─ string.h
| | └─ string.h:Zone.Identifier
| | └─ uart.h
| | └─ uart.h:Zone.Identifier
| | └─ vga.h
| | └─ vga.h:Zone.Identifier
| | └─ vsprintf.h
| | └─ vsprintf.h:Zone.Identifier
| | └─ wallClock.h
| | └─ wallClock.h:Zone.Identifier
| └─ kernel
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ mem
| | | └─ Makefile
| | | └─ Makefile:Zone.Identifier
| | | └─ dPartition.c
| | | └─ dPartition.c:Zone.Identifier
| | | └─ eFPartition.c
| | | └─ eFPartition.c:Zone.Identifier
| | | └─ malloc.c
| | | └─ malloc.c:Zone.Identifier
| | | └─ pMemInit.c
| | | └─ pMemInit.c:Zone.Identifier
| | └─ tick.c
| | └─ tick.c:Zone.Identifier
| | └─ wallClock.c
| | └─ wallClock.c:Zone.Identifier
| └─ lib
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ string.c
| | └─ string.c:Zone.Identifier
| └─ myOS.ld
| └─ myOS.ld:Zone.Identifier
| └─ osStart.c
| └─ osStart.c:Zone.Identifier
| └─ printk
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ myPrintk.c
| | └─ myPrintk.c:Zone.Identifier
| | └─ types.h
| | └─ types.h:Zone.Identifier
| | └─ vsprintf.c
| | └─ vsprintf.c:Zone.Identifier
| └─ start32.s

```

```

|   ├── start32.S:Zone.Identifier
|   ├── userInterface.h
|   └── userInterface.h:Zone.Identifier
└── output
    ├── multibootheader
    │   └── multibootHeader.o
    ├── myOS
    │   ├── dev
    │   │   ├── i8253.o
    │   │   ├── i8259A.o
    │   │   ├── uart.o
    │   │   └── vga.o
    │   ├── i386
    │   │   ├── io.o
    │   │   ├── irq.o
    │   │   └── irqs.o
    │   ├── kernel
    │   │   ├── mem
    │   │   │   ├── dPartition.o
    │   │   │   ├── eFPartition.o
    │   │   │   ├── malloc.o
    │   │   │   └── pMemInit.o
    │   │   ├── tick.o
    │   │   └── wallClock.o
    │   ├── lib
    │   │   └── string.o
    │   ├── osStart.o
    │   ├── printk
    │   │   └── myPrintk.o
    │   └── start32.o
    ├── myOS.elf
    └── userApp
        ├── main.o
        ├── memTestCase.o
        └── shell.o
└── source2run.sh
└── source2run.sh:Zone.Identifier
└── userApp
    ├── Makefile
    ├── Makefile:Zone.Identifier
    ├── main.c
    ├── main.c:Zone.Identifier
    ├── memTestCase.c
    ├── memTestCase.c:Zone.Identifier
    ├── memTestCase.h
    ├── memTestCase.h:Zone.Identifier
    ├── shell.c
    ├── shell.c:Zone.Identifier
    ├── shell.h
    └── shell.h:Zone.Identifier

```

21 directories, 122 files

输入./source2run.sh 指令后，编译，链接，生成 myOS.elf 文件。输入指令 qemu-system-i386 -kernel myOS.elf -serial pty & 运行。

使用 screen 命令进入交互界面（与 QEMU）`sudo screen /dev/pts/x`（x为生成的对应值）

第四部分 运行结果

1.help

```
Student >:help
help
USAGE: help [cmd]

list all registered commands:
command name: description
    cmd: list all registered commands
    help: help [cmd]
testMalloc1: Malloc, write and read.
testMalloc2: Malloc, write and read.
maxMallocSizeNow: MAX_MALLOC_SIZE always changes. What's the value Now?
    testdP1: Init a dPation(size=0x100) [Alloc,Free]* with step = 0x20
    testdP2: Init a dPation(size=0x100) A:B:C:- ==> -:B:C:- ==> -:C:- ==> - .
    testdP3: Init a dPation(size=0x100) A:B:C:- ==> A:B:- ==> A:- ==> - .
    testeFP: Init a eFPation. Alloc all and Free all.
```

2.testmalloc

```
Student >:maxMallocSizeNow
maxMallocSizeNow
MAX_MALLOC_SIZE: 0x7efb000 (with step = 0x1000);
Student >:testMalloc1
testMalloc1
We allocated 2 buffers.
BUF1(size=19, addr=0x105a98) filled with 17(*): *****
BUF2(size=24, addr=0x105ab8) filled with 22(#): #####

Student >:testMalloc2
testMalloc2
We allocated 2 buffers.
BUF1(size=9, ANKOWN command: [Allled with 9(+): ++++++++
BUF2(size=19, addr=0x105ab0) filled with 19(,): ,,,,,,,,,,,,,,
tudent >:A
Student >:maxMallocSizeNow
maxMallocSizeNow
MAX_MALLOC_SIZE: 0x7efb000 (with step = 0x1000);
Student >:
```

前后MAX MALLOC SIZE不变, 内存成功分配并释放

3.testdP1

```
Student >:testdP1
testdP1
We had successfully malloc() a small memBlock (size=0x100, addr=0x105a98);
It is initialized as a very small dPartition;
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0xf0, nextStart=0x0)
Alloc a memBlock with size 0x10, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x20, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x40, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x80, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x100, failed!
Now, converse the sequence.
Alloc a memBlock with size 0x100, failed!
Alloc a memBlock with size 0x80, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x40, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x20, success(addr=0x105aa8)!.....Relaesed;
Alloc a memBlock with size 0x10, success(addr=0x105aa8)!.....Relaesed;
Student >:█
```

一个 dPartition 实际可分配的最大连续空间为:

$\text{totalSize} - \text{dPartition_size} - \text{EMB_size} = 0x100 - 0x8 - 0x8 = 0xF0;$

对齐到 8 字节后, 申请 0x10、0x20、0x40、0x80 都小于 0xF0, 成功分配, addr始终不变说明成功释放;

申请 0x100 (256 > 240) 超过了最大可用 0xF0, 导致分配失败

4.testdP2

```
Student >:testdP2
testdP2
We had successfully malloc() a small memBlock (size=0x100, addr=0x105a98);
It is initialized as a very small dPartition;
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0xf0, nextStart=0x0)
Now, A:B:C:- ==> -:B:C:- ==> -:C- ==> - .
Alloc memBlock A with size 0x10: success(addr=0x105aa8)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ab8)
EMB(start=0x105ab8, size=0xd8, nextStart=0x0)
Alloc memBlock B with size 0x20: success(addr=0x105ac0)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ae0)
EMB(start=0x105ae0, size=0xb0, nextStart=0x0)
Alloc memBlock C with size 0x30: success(addr=0x105ae8)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105b18)
EMB(start=0x105b18, size=0x78, nextStart=0x0)
Now, release A.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0x10, nextStart=0x105b18)
EMB(start=0x105b18, size=0x78, nextStart=0x0)
Now, release B.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0x38, nextStart=0x105b18)
EMB(start=0x105b18, size=0x78, nextStart=0x0)
At last, release C.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0xf0, nextStart=0x0)
```

1. 初始化阶段，在地址 0x105A98 处创建 dPartition 结构体，紧跟一个 EMB 结构。空闲块大小为 $\text{ALIGN8}(0x100 - 8 - 8) = 0xF0$ ，首个空闲块起始地址为 0x105AA0。
2. 分配 A，分配请求 $\text{size}=0x10$ ，在首个空闲块 ($\text{size}=0xF0$) 上进行分裂，生成 EMB A (头部在 0x105AA0, $\text{size}=0x10$)，剩余空闲 EMB 在 0x105AB8, $\text{size}=\text{ALIGN8}(0xF0-8-0x10)=0xD8$ ，更新 firstFreeStart 为 0x105AB8。
3. 分配 B，分配请求 $\text{size}=0x20$ ，在当前首个空闲块 ($\text{size}=0xD8$) 上进行分裂，生成 EMB B (头部在 0x105AB8, $\text{size}=0x20$)，剩余空闲 EMB 在 0x105AE0, $\text{size}=\text{ALIGN8}(0xD8-8-0x20)=0xB0$ ，更新 firstFreeStart 为 0x105AE0。
4. 分配 C，分配请求 $\text{size}=0x30$ ，在当前首个空闲块 ($\text{size}=0xB0$) 上进行分裂，生成 EMB C (头部在 0x105AE0, $\text{size}=0x30$)，剩余空闲 EMB 在 0x105B18, $\text{size}=\text{ALIGN8}(0xB0-8-0x30)=0x78$ ，更新 firstFreeStart 为 0x105B18。
5. 释放 A，计算 EMB A 头地址 $\text{embAddr}=0x105AA0$ ，将其插入链表头部，构成 $\text{EMB}@0x105AA0 \rightarrow \text{EMB}@0x105B18$ ，无相邻块合并。
6. 释放 B，计算 EMB B 头地址 $\text{embAddr}=0x105AB8$ ，将其插入 0x105AA0 与 0x105B18 之间，检测到与前驱 $\text{EMB}@0x105AA0$ 相邻，合并为 $\text{EMB}@0x105AA0$ ($\text{size}=0x10+8+0x20=0x38$)，链表变为 $\text{EMB}@0x105AA0 \rightarrow \text{EMB}@0x105B18$ 。
7. 释放 C，计算 EMB C 头地址 $\text{embAddr}=0x105AE0$ ，将其插入 $\text{EMB}@0x105AA0$ 与 $\text{EMB}@0x105B18$ 之间，先与后续 $\text{EMB}@0x105B18$ 相邻合并 ($\text{size}=0x30+8+0x78=0xB0$)，再与前驱 $\text{EMB}@0x105AA0$ 相邻合并 ($\text{size}=0x38+8+0xB0=0xF0$)，最终链表为 $\text{EMB}@0x105AA0$ ($\text{size}=0xF0$)。

结果与预期一致。

5.testdP3

```
testdP3
We had successfully malloc() a small memBlock (size=0x100, addr=0x105a98);
It is initialized as a very small dPartition;
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0xf0, nextStart=0x0)
Now, A:B:C:- ==> -:B:C:- ==> -:C- ==> - .
Alloc memBlock A with size 0x10: success(addr=0x105aa8)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ab8)
EMB(start=0x105ab8, size=0xd8, nextStart=0x0)
Alloc memBlock B with size 0x20: success(addr=0x105ac0)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ae0)
EMB(start=0x105ae0, size=0xb0, nextStart=0x0)
Alloc memBlock C with size 0x30: success(addr=0x105ae8)!
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105b18)
EMB(start=0x105b18, size=0x78, nextStart=0x0)
At last, release C.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ae0)
EMB(start=0x105ae0, size=0xb0, nextStart=0x0)
Now, release B.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105ab8)
EMB(start=0x105ab8, size=0xd8, nextStart=0x0)
Now, release A.
dPartition(start=0x105a98, size=0x100, firstFreeStart=0x105aa0)
EMB(start=0x105aa0, size=0xf0, nextStart=0x0)
Student >: 
```

测试的I/O信息写错了，实际过程为A:B:C:- ==> A:B:- ==> A:- ==> - .

1. 初始化阶段，在地址 0x105A98 处创建 dPartition 结构体，紧跟一个 EMB 结构。空闲块大小为 $\text{ALIGN8}(0x100 - 8 - 8) = 0xF0$ ，首个空闲块起始地址为 0x105AA0。
2. 分配 A，分配请求 $\text{size}=0x10$ ，在首个空闲块 ($\text{size}=0xF0$) 上进行分裂，生成 EMB A (头部在 0x105AA0, $\text{size}=0x10$)，剩余空闲 EMB 在 0x105AB8, $\text{size}=\text{ALIGN8}(0xF0-8-0x10)=0xD8$ ，更新 firstFreeStart 为 0x105AB8。
3. 分配 B，分配请求 $\text{size}=0x20$ ，在当前首个空闲块 ($\text{size}=0xD8$) 上进行分裂，生成 EMB B (头部在 0x105AB8, $\text{size}=0x20$)，剩余空闲 EMB 在 0x105AE0, $\text{size}=\text{ALIGN8}(0xD8-8-0x20)=0xB0$ ，更新 firstFreeStart 为 0x105AE0。
4. 分配 C，分配请求 $\text{size}=0x30$ ，在当前首个空闲块 ($\text{size}=0xB0$) 上进行分裂，生成 EMB C (头部在 0x105AE0, $\text{size}=0x30$)，剩余空闲 EMB 在 0x105B18, $\text{size}=\text{ALIGN8}(0xB0-8-0x30)=0x78$ ，更新 firstFreeStart 为 0x105B18。
5. 释放 C，计算 EMB C 头部地址 0x105AE0，将其插入链头，暂时形成 $\text{EMB}@0x105AE0$ ($\text{size}=0x30$) $\rightarrow$ $\text{EMB}@0x105B18$ ($\text{size}=0x78$)；检测到 $\text{freedEnd} = 0x105AE0 + 8 + 0x30 = 0x105B18$ 与后继相邻，合并为 $\text{EMB}@0x105AE0$ ，size 字段更新为 $0x30 + 8 + 0x78 = 0xB0$ ，链表恢复为单节点 $\text{firstFreeStart}=0x105AE0$ 。
6. 释放 B，计算 EMB B 头 $\text{Addr}=0x105AB8$ ，将其插入链头， $\text{EMB}@0x105AB8$ ($\text{size}=0x20$) $\rightarrow$ $\text{EMB}@0x105AE0$ ($\text{size}=0xB0$)；检测到 $\text{freedEnd} = 0x105AB8 + 8 + 0x20 = 0x105AE0$ ，与后继相邻，合并为 $\text{EMB}@0x105AB8$ ， $\text{size}=0x20 + 8 + 0xB0 = 0xD8$ ，链表恢复为单节点 $\text{firstFreeStart}=0x105AB8$ 。
7. 释放 A，计算 EMB A 头 $\text{Addr}=0x105AA0$ ，将其插入链头， $\text{EMB}@0x105AA0$ ($\text{size}=0x10$) $\rightarrow$ $\text{EMB}@0x105AB8$ ($\text{size}=0xD8$)，检测到 $\text{freedEnd} = 0x105AA0 + 8 + 0x10 = 0x105AB8$ ，与后继相邻，合并为 $\text{EMB}@0x105AA0$ ， $\text{size}=0x10 + 8 + 0xD8 = 0xF0$ ，链表恢复为单节点 $\text{firstFreeStart}=0x105AA0$ 。

结果与预期一致。

6.testeFP

```
Student >:testeFP
testeFP
X:0x105a98:160
We had successfully malloc() a small memBlock (size=0xa0, addr=0x105a98);
It is initialized as a very small ePartition;
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105aa8)
EEB(start=0x105aa8, next=0x105acc)
EEB(start=0x105acc, next=0x105af0)
EEB(start=0x105af0, next=0x105b14)
EEB(start=0x105b14, next=0x0)
Alloc memBlock A, start = 0x105aac: 0xaaaaaaaa
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105acc)
EEB(start=0x105acc, next=0x105af0)
EEB(start=0x105af0, next=0x105b14)
EEB(start=0x105b14, next=0x0)
Alloc memBlock B, start = 0x105ad0: 0xbbbbbbbb
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105af0)
EEB(start=0x105af0, next=0x105b14)
EEB(start=0x105b14, next=0x0)
Alloc memBlock C, start = 0x105af4: 0xcccccccc
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105b14)
EEB(start=0x105b14, next=0x0)
Alloc memBlock D, start = 0x105b18: 0xdddddddd
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x0)
Alloc memBlock E, failed!
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x0)
Now, release A.
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105aa8)
EEB(start=0x105aa8, next=0x0)
Now, release B.
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105acc)
EEB(start=0x105acc, next=0x105aa8)
EEB(start=0x105aa8, next=0x0)
Now, release C.
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105af0)
EEB(start=0x105af0, next=0x105acc)
EEB(start=0x105acc, next=0x105aa8)
EEB(start=0x105aa8, next=0x0)
Now, release D.
eFPartition(start=0x105a98, totalN=0x4, perSize=0x20, firstFree=0x105b14)
EEB(start=0x105b14, next=0x105af0)
EEB(start=0x105af0, next=0x105acc)
EEB(start=0x105acc, next=0x105aa8)
EEB(start=0x105aa8, next=0x0)
Student >:
```

1. 初始化时，eFPartition 头部（占用对齐后 12 字节）位于 0x105A98，紧接着从 0x105AA8 开始依次生成了四个空闲块描述符 EEB，此时 eFPartition.totalN=4，perSize=ALIGN8(0x20)=0x20，firstFree=0x105AA8。
2. 分配 A 时从 0x105AA8 摘除其 EEB，用户可用地址为 $0x105AA8 + 4 = 0x105AAC$ ，分配后将 firstFree 更新为 0x105ACC。
3. 分配 B 时从 0x105ACC 摘除其 EEB，用户可用地址为 $0x105ACC + 4 = 0x105AD0$ ，分配后将 firstFree 更新为 0x105AF0。
4. 分配 C 时从 0x105AF0 摘除其 EEB，用户可用地址为 $0x105AF0 + 4 = 0x105AF4$ ，分配后将 firstFree 更新为 0x105B14。

5. 分配 D 时从 0x105B14 摘除其 EEB，用户可用地址为 $0x105B14 + 4 = 0x105B18$ ，分配后将 firstFree 更新为 0x0，表示空闲链表已空。
6. 尝试分配 E 时，由于 firstFree 已为 0，分配失败。
7. 释放 A 时从用户地址 0x105AAC 计算得到其 EEB 头部地址 0x105AA8，将该 EEB 插入空闲链表头部，firstFree 从 0x0 更新为 0x105AA8。
8. 释放 B 时从用户地址 0x105AD0 计算得到其 EEB 头部地址 0x105ACC，将该 EEB 插入空闲链表头部，firstFree 从 0x105AA8 更新为 0x105ACC。
9. 释放 C 时从用户地址 0x105AF4 计算得到其 EEB 头部地址 0x105AF0，将该 EEB 插入空闲链表头部，firstFree 从 0x105ACC 更新为 0x105AF0。
10. 释放 D 时从用户地址 0x105B18 计算得到其 EEB 头部地址 0x105B14，将该 EEB 插入空闲链表头部，firstFree 从 0x105AF0 更新为 0x105B14。

结果与预期一致。